Total Recall at What Cost? Benchmarking the Serving Cost of Agentic Memory Systems

Natchanon Pollertlam
natchanon.p@brickstech.co
Bricks Technology
Thailand

Witchayut Kornsuwannawit
witchayut.k@brickstech.co
Bricks Technology
Thailand

Abstract

Long-running conversational agents increasingly rely on a memory system to avoid resending the whole conversation each turn, yet how much that costs to serve has received little systematic benchmarking. We compare three memory systems (Mem0, Hindsight, and Mastra Observational Memory) against two reference strategies — a fixed-size rolling window and resubmitting the full transcript — across two backbones and conversations of up to 400 turns, pairing every cost measurement with answer accuracy on 665 LoCoMo questions. First, a memory system's serving cost cannot be predicted from conversation length and message size alone: a regression that tracks the two reference strategies closely misses the memory systems by 18–69%, their cost driven instead by internal memory behavior. Second, a break-even analysis shows that whether — and when — a memory system becomes cheaper to serve than the full transcript is highly sensitive to the system and the backbone, from the first tens of turns for the cheapest to never within 400 turns for the most expensive. Third, no system wins on both axes: accuracy spans 21–54%, and the backbone choice drives cost as much as the memory system does.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; **Distributed artificial intelligence**; • **General and reference**;

Keywords

agentic memory systems, LLM inference cost, cost benchmarking, cost modeling, cost break-even, long-running conversational agents, cost–accuracy trade-off

1 Introduction

Conversational agents built on large language models (LLMs) have moved from single-session demonstrations into deployments that span weeks or months of interaction. To stay coherent across sessions, such an agent must reuse information established in earlier turns. Two strategies address this. The first resubmits the full prior transcript into the context window of a long-context LLM on every turn, relying on the model to attend over all past interaction. The second builds a dedicated *memory system* that distills past interaction into compact records — extracted facts, summaries, or knowledge-graph entries — and retrieves only the relevant subset at query time [6, 15, 23]. The second strategy has since been developed into a variety of architectures.

As these agents scale, deployment cost becomes a first-order concern alongside accuracy. Commercial LLM APIs price requests by token count [2, 21], so, because the history grows with every turn, the per-turn cost of resubmitting it rises without bound as a session continues. Memory systems are adopted, in large part, to escape this growth: by replacing an ever-larger transcript with a small retrieved payload, they promise a per-turn cost that stays roughly flat as a conversation lengthens. This framing, however, overlooks the cost of the memory system itself. Modern memory systems are not passive stores; they run their own LLM pipelines — extracting facts at ingest, embedding and retrieving them, and in some designs periodically reflecting over accumulated state to consolidate it. Each stage incurs its own billable model calls. Yet evaluation of memory systems concentrates almost entirely on accuracy and recall [11, 17, 29], and what cost evidence exists appears as isolated token-savings or latency figures reported by individual systems under their own conditions. No study has measured the serving cost of memory systems across systems, under a common backbone and pricing, against a transparent baseline.

As a result, a practitioner choosing a memory system today cannot answer a basic question: what does it cost to serve, and does that cost actually beat resubmitting the transcript? A memory system's per-turn cost is shaped by its own internal pipeline behavior, and whether it is economical further depends on how long the conversation runs and which backbone model serves it. Without a controlled, cross-system measurement, the cost case for memory systems rests on assumption rather than evidence.

In this study, we benchmark the serving cost of agentic memory systems. We measure three memory systems — Mem0 [6], Hindsight [14], and Mastra Observational Memory [3] — against two reference strategies that bracket the cost surface: a fixed-size rolling window and full-transcript resubmission. We run every system across two backbone models at two reasoning-effort settings, on synthetic conversations of up to 400 turns, and pair each cost measurement with answer accuracy on the LoCoMo benchmark [17] so that cost and accuracy are read from a single matched configuration. Our contributions are:

- A controlled benchmark of the serving cost of three agentic memory systems, measured against two transparent reference strategies: a rolling window and full-transcript resubmission.
- A per-turn cost model that fits message size and conversation depth as two independent terms and is validated on held-out workloads; it predicts the reference strategies but not the memory systems, whose cost is driven by internal memory state.
- A break-even analysis that identifies when a memory system becomes cheaper to serve than resubmitting the full transcript.
- A joint cost–accuracy comparison on LoCoMo across memory systems and backbone models.

2 Related Work

Memory systems for conversational agents. Retrieval-augmented generation [15] is an early form of memory augmentation, prepending retrieved document chunks to the prompt to ground generation in external knowledge. MemGPT [23] draws an analogy between LLM context management and operating-system virtual memory, paging facts between an in-context working memory and external long-term storage. Mem0 [6] extracts atomic, flat-typed facts from each turn and stores them in a vector database, retrieving the top-k at query time. Later systems pursue richer representations, including interlinked memory notes [30], temporally-aware knowledge graphs [24], and durative summaries of temporally continuous facts [27]. The three systems we benchmark span this design space: Mem0 represents flat extract-and-retrieve, Hindsight [14] adds a retain–recall–reflect memory ingestion pipeline, and Mastra Observational Memory [3] runs an observer–reflector–actor loop with threshold-triggered consolidation. Crucially, these architectures differ not only in what they store but in *how they spend compute* — a fixed-size fact extraction, a retrieval payload that grows as the conversation lengthens, or a threshold-triggered memory-consolidation pass — so they cannot be assumed to share a single cost profile. Prior work characterizes these systems by their representations and recall accuracy; we characterize them by serving cost.

Benchmarking memory and long context. Several benchmarks evaluate how well a system recalls extended interaction. LoCoMo [17] provides multi-session dialogues whose questions span single-hop, multi-hop, temporal, and open-domain categories; LongMemEval [29] tests information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention; and PersonaMem [11] probes persona consistency across questions. Such benchmarks have also been used to weigh memory systems against long-context inference directly, as expanding context windows raise the question of whether retrieval remains necessary [13, 25], even though long-context models attend unevenly across a long prompt [16]. That debate, however, has been conducted almost entirely on accuracy: these benchmarks measure what a system recalls, not what it costs to serve, and the serving cost of a memory system is left uninstrumented. Our work pairs accuracy on LoCoMo with a matched cost measurement so that the two are directly comparable.

Cost and efficiency of LLM inference. Per-token API pricing makes inference cost a central production concern, and a range of techniques target it. Prompt caching reuses the precomputed key-value states of a shared input prefix [9], and providers discount cached input tokens steeply [1, 20]; prompt compression instead shortens the input itself [12]. Such techniques optimize a *single* inference path. A memory system, by contrast, is a multi-stage pipeline whose total billed cost compounds an ingest stage, a retrieval stage, and an answer stage, each issuing its own model calls. The compositional cost of such a pipeline — and the conversation length at which it undercuts simply resubmitting the transcript — has not been characterized. We provide that characterization: a controlled serving-cost benchmark of memory systems against transparent floor and ceiling baselines, with a break-even analysis and a matched accuracy comparison.

3 Methodology

We evaluate three memory systems in two separate benchmarks that share a common backbone, reasoning-effort, and embedding configuration: a **cost** benchmark, measuring billable serving cost as a function of conversation length and turn size, and an **accuracy** benchmark, measuring answer correctness on a stratified subset of a multi-session conversational QA benchmark. Section 3.5 combines them into a single cost-per-correct-answer statistic.

3.1 Memory Systems

We benchmark three memory systems: **Mem0** [6], **Hindsight** [14], and **Mastra Observational Memory** (Mastra OM) [3]. Per-system pipeline configuration is tabulated in Table 5.

We additionally evaluate two reference strategies that bracket the cost surface:

Rolling window (floor). Each turn sees only the last 10 turns; nothing is stored persistently — the cheapest possible strategy.

Full history (ceiling). The entire transcript is resubmitted every turn. Every memory system's break-even point (Section 4.2) is measured against this ceiling.

3.2 Backbone Models and Reasoning Levels

All systems are evaluated under the same two backbones, **gpt-oss-20b** [19] and **Gemma 4 26B A4B** [10], each at two reasoning-effort settings (*low*, *medium*). Embeddings use `pplx-embed-v1-0.6b` [8]; per-token pricing and OpenRouter provider routing are given in Appendix A.

3.3 Cost Benchmark Design

Figure 1 gives an end-to-end overview of the cost benchmark, from the design grid through the fitted cost model; the remainder of this subsection details each stage.

Grid. For each (system, model) pair we sample five (N, L) cells: the four corners and the center of a grid over conversation length N and per-turn token size L. Each cell is run eight times with different random seeds. The exact (N, L) values, including extra cells for the baselines and Mastra OM, are listed in Appendix B.

Synthetic dialogue generation. Each cost-benchmark dialogue is generated synthetically by an LLM: for each (N, L, seed) the model produces a two-speaker conversation of N turns at roughly L tokens per turn, prompted to keep concrete personal detail — names, dates, preferences, plans. Over-length turns are trimmed and under-length turns padded so turn lengths stay close to L; each finished dialogue is cached so it is identical across repeats and across systems. Generating dialogues lets us fill every (N, L) cell exactly. The generation prompt is reproduced in Appendix C.

Cost model. For a turn at depth t in a conversation whose messages average L tokens, let C be the total LLM input tokens billed across ingest, retrieval, and answer. We model C rather than dollar cost: dollar cost follows from the per-token rates of Appendix A, and output tokens, which do not grow with depth, we track separately through the γ diagnostic below.

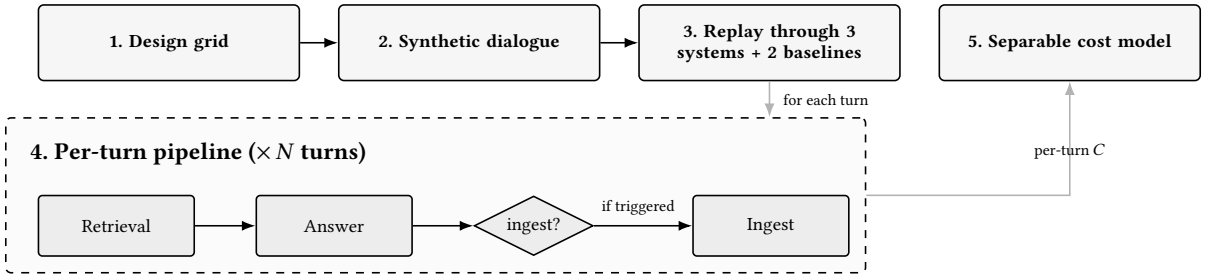

Figure 1: Overview of the cost benchmark. (1) We sample five (N, L) cells: the four corners and the center of a grid over conversation length N and per-turn token size L. (2) For each (N, L, seed), an LLM generates a two-speaker conversation that is cached and reused unchanged, so every system sees the same input. (3) Each cached dialogue is replayed through three memory systems and two reference baselines. (4) Within a run, every turn passes through retrieval, answer, and an ingest gate that decides whether to flush the buffered turns; the input tokens billed across these three stages give the per-turn cost C. (5) We then fit a log-log cost model with separate N and L terms, one fit per (system, model), validated by leave-one-out cross-validation with bootstrap confidence intervals.

Conversation depth and message size scale cost differently, so we fit them as two separate log-log terms rather than as one cumulative-content predictor ($N{\cdot}L$):

$$\log(C{+}1) = a + p\,\log(L{+}1) + q\,\log(t{+}1), \tag{1}$$

where p is how fast cost grows with message size and q how fast it grows with depth: $q \approx 0$ is the signature of a bounded context window; $p \approx q \approx 1$ is cumulative content. We fit this form once per (system, model). Alongside it we report three token-accounting diagnostics: γ (output-to-input ratio), ζ_{ans} (answer-stage reasoning-to-output ratio), and ζ_{ing} (ingest-stage reasoning-to-output ratio).

Held-out validation. We test whether each fitted model generalizes by leave-one-cell-out cross-validation [26]: for every (system, model) we drop one (N, L) cell, refit Equation (1) on the rest, and predict the dropped cell's mean per-turn cost. The error measure, LOOCV-MAPE, is the mean absolute percentage error over the held-out cells. We use it as a diagnostic, not a pass/fail bar. A low value means message size and depth alone account for the system's cost. A high value is itself a finding: it shows that the system's cost is driven by internal memory state that (L, t) cannot capture (Section 4.1).

Confidence intervals. Confidence intervals for p and q come from a cluster bootstrap [5, 7] with B=1,000 resamples. A *run* is one end-to-end execution of a synthetic dialogue through the system; each cell contributes 8 runs at different seeds. The bootstrap resamples whole runs, not individual turns, because turns within a run share state — transcript, memory store, observation buffer — and are not independent.

3.4 Accuracy Benchmark Design

Dataset. Accuracy is evaluated on **LoCoMo**, restricted to a stratified subset — four full dialogues, 665 evaluated QA pairs — chosen so its question-category mix matches the full corpus (Appendix B). Restricting accuracy evaluation to a single corpus anchors the joint cost–accuracy analysis on one distribution.

Protocol. For each (system, setting) cell, the system ingests the full conversation, retrieves from its memory at query time, and produces a free-form or multiple-choice answer at temperature=0.7 with a single pass per question.

Judge. The judge is **gpt-oss-120b** at temperature=0 with a single pass per question. We hold it fixed across all setting cells so judging is consistent across systems. Accuracy is the fraction of questions judged correct, reported per (system, setting) cell.

3.5 Joint Cost–Accuracy Framing

For each system at each setting we have a matched $(\hat{C}, \text{Acc})$ pair, where $\hat{C}$ is the mean billable cost at the (N=100, L=100) reference cell and Acc is the accuracy on the LoCoMo subset. We report this joint matrix (Table 3) and the cost-per-correct-answer ratio $\hat{C}/\text{Acc}$ as the primary derived statistic.

4 Results

We report four things: (i) the per-turn cost model and its fitted exponents for the three memory systems and two baselines, (ii) held-out cross-validation of that model, (iii) a cost break-even analysis of when a memory system becomes cheaper to serve than the full-history baseline, and (iv) accuracy on the LoCoMo subset. We then present the joint cost–accuracy matrix.

4.1 Per-Turn Cost Model

Table 1 reports the fitted exponents of the separable cost model (Equation (1)), its in-sample R^2, and held-out LOOCV-MAPE. We summarize each as a range across the four (backbone, reasoning-effort) settings. Table 6 gives the per-(system, setting) breakdown.

Fitted exponents. The two baselines set the reference points. The full-history baseline fits $p \approx q \approx 1$ ($R^2 \geq 0.996$): every turn resubmits the whole transcript, so cost is proportional to message size times depth. The rolling-window baseline fits $p \approx 0.9$ but $q \approx 0.1$: cost scales almost linearly with message size but is nearly flat in depth. This is the result of a bounded ten-turn window. A single cumulative-content predictor $T = N{\cdot}L$ cannot capture this depth–size split. Fit on $\log(T{+}1)$ alone, the same rolling-window data reaches only $R^2 = 0.39$–0.40, against the 0.91–0.95 of the

System	p (L)	q (t)	R^2	LOOCV-MAPE
Full history	0.95–0.97	0.94–0.97	0.996–0.999	0.029–0.053
Rolling window (k=10)	0.85–0.92	0.10–0.12	0.913–0.950	0.061–0.065
Mem0	0.16–0.18	0.07–0.09	0.048–0.069	0.184–0.222
Hindsight	0.14–0.17	0.41–0.43	0.320–0.348	0.461–0.478
Mastra OM	0.61–0.79	0.23–0.36	0.457–0.557	0.408–0.685

Table 1: Separable per-turn cost model $\log(C{+}1) = a + p\log(L{+}1) + q\log(t{+}1)$ fitted per (system, setting), summarized as ranges across the four settings. R^2 is in-sample; LOOCV-MAPE is leave-one-(N, L)-cell-out held-out error. Held-out folds per row: full-history 8, rolling-window 8, Mastra OM 7, Mem0 5, Hindsight 5. Table 6 gives the per-(system, setting) breakdown.

separable model. The three memory systems show different cost patterns. Mem0 has small exponents on both axes ($p{\approx}0.17$, $q{\approx}0.08$). Hindsight has a small message-size exponent but the largest depth exponent ($p{\approx}0.15$, $q{\approx}0.42$). Mastra OM has the largest message-size exponents of the three ($p \in [0.61, 0.79]$, $q \in [0.23, 0.36]$) and the highest in-sample R^2.

Held-out validation. LOOCV-MAPE separates the five configurations into two clear groups. The two baselines generalize to within 2.9–6.5%. The memory systems do not: 18–22% for Mem0, 46–48% for Hindsight, and 41–69% for Mastra OM. The worst cell is off by 1.9× the true cost. The held-out failures cluster at the grid corners: short conversations for Hindsight and small-message (L=50) cells for Mastra OM. We return to this split in Section 5.

Token-accounting diagnostics. Table 7 reports three per-stage ratios that the cost model does not capture on its own: the output-to-input token ratio γ, the answer-stage reasoning-to-output ratio ζ_{ans}, and the ingest-stage reasoning-to-output ratio ζ_{ing}. These ratios, not the cost exponents, drive the cross-backbone cost gap discussed in Section 5.

4.2 Cost Break-Even: When a Memory System Pays for Itself

A memory system pays for itself only once a conversation is long enough to spread its overhead. We define the *break-even length* as the turn at which a system's cumulative cost drops below the full-history cost and stays below it. We compute break-even from measured per-turn cost (8 reps averaged, no fitted-model extrapolation). Each value therefore comes from real grid cells and is not affected by the held-out prediction failure of Section 4.1.

Figure 2 traces this for a 400-turn conversation under gpt-oss-20b: Mastra OM breaks even at turn 0, Mem0 at turn 82, and Hindsight only at turn 356. Across all measured 400-turn cells the break-even length spans Mastra OM 0–86, Mem0 0–342, and Hindsight 60–never. By a 400-turn conversation the full transcript costs up to 12.7× a memory system that has broken even. Hindsight at small messages, in turn, can cost up to 3.3× the full transcript. Break-even arrives earlier with larger per-turn messages and on Gemma 4 26B A4B. We discuss these workload dependencies in Section 5.

4.3 Accuracy on LoCoMo

Table 2 reports per-(system, setting) accuracy on the 665 evaluated questions of the LoCoMo subset. Mem0 and Mastra OM cover a wide range, $[0.21, 0.52]$, and both score higher on Gemma 4 26B A4B than on gpt-oss-20b. We report Hindsight's per-cell accuracy here. But because its ingest stage did not run under the per-cell backbone (Section 5.4), we do not compare its cells across backbones or reasoning levels.

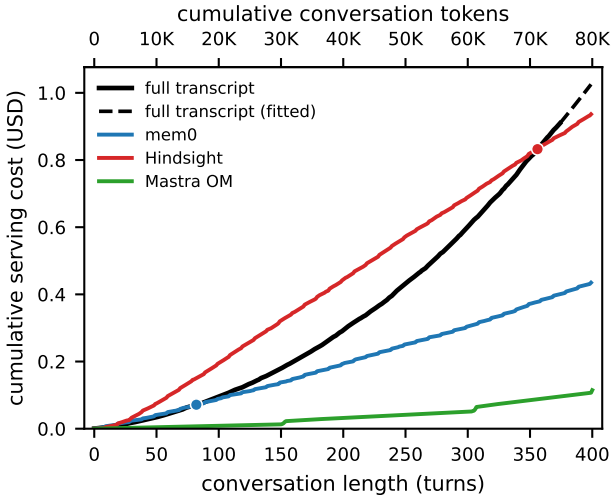

Figure 2: Cumulative serving cost vs. conversation length for a 400-turn conversation at 200 tokens per turn under `gpt-oss-20b` at low reasoning effort (top axis: cumulative conversation tokens). Dots mark the break-even turn at which a system's cumulative cost drops below the full-history ceiling; the full-history curve is measured to turn 374 and fitted (dashed) beyond (Section 5.4).

4.4 Joint Cost–Accuracy Matrix

Table 3 pairs each cell's accuracy with its mean billable cost at the reference cell (N=100, L=100), in USD per 100-turn conversation (8 reps averaged), and with the cost-per-correct-answer ratio $\hat{C}/\text{Acc}$. We discuss the trade-off in Section 5.

At the reference cell (Table 3), most memory systems have not yet paid back their ingest cost against full history. Table 4 recomputes the matrix at (N=400, L=200). There, all Mem0 and Mastra OM cells, and all Hindsight cells except gpt-oss-20b medium, have crossed break-even (Table 8). Hindsight on gpt-oss-20b medium stays slightly above the measured full-history cost ($0.869 vs. $0.858) and is marked *never*. Accuracy is measured once per cell on the

System	gpt-oss-20b low	gpt-oss-20b medium	Gemma 4 26B A4B low	Gemma 4 26B A4B medium
Mem0	0.322 [0.287, 0.358]	0.214 [0.184, 0.246]	0.498 [0.460, 0.536]	0.516 [0.478, 0.554]
Mastra OM	0.361 [0.325, 0.398]	0.308 [0.274, 0.344]	0.429 [0.391, 0.466]	0.502 [0.464, 0.540]
Hindsight†	0.528 [0.490, 0.565]	0.541 [0.503, 0.579]	0.493 [0.455, 0.531]	0.498 [0.460, 0.536]

Table 2: Accuracy on the LoCoMo subset (665 evaluated questions), per (system, setting) cell. Bracketed values are Wilson 95% confidence intervals [4, 28] at n=665; they treat the 665 questions as independent and so understate uncertainty given dialogue-level clustering (Section 5.4). †Hindsight's ingest stage ran under a configuration the benchmark did not control, so its cells are not comparable across the backbone/reasoning columns (Section 5.4).

System	Metric	gpt-oss-20b low	gpt-oss-20b medium	Gemma 4 26B A4B low	Gemma 4 26B A4B medium
Mem0	Acc	0.322 [0.287, 0.358]	0.214 [0.184, 0.246]	0.498 [0.460, 0.536]	0.516 [0.478, 0.554]
	$\hat{C}$ (USD)	$0.059	$0.065	$0.019	$0.019
	$\hat{C}$/Acc	0.183	0.305	0.038	0.037
Mastra OM	Acc	0.361 [0.325, 0.398]	0.308 [0.274, 0.344]	0.429 [0.391, 0.466]	0.502 [0.464, 0.540]
	$\hat{C}$ (USD)	$0.010	$0.044	$0.030	$0.031
	$\hat{C}$/Acc	0.028	0.143	0.070	0.062
Hindsight†	Acc	0.528 [0.490, 0.565]	0.541 [0.503, 0.579]	0.493 [0.455, 0.531]	0.498 [0.460, 0.536]
	$\hat{C}$ (USD)	$0.240	$0.243	$0.051	$0.052
	$\hat{C}$/Acc	0.455	0.450	0.104	0.104

Table 3: Joint cost–accuracy matrix at the (N=100, L=100) reference cell. $\hat{C}$ is mean billable cost in USD per 100-turn conversation over 8 reps, priced at the rates in Appendix A; Acc is from the LoCoMo subset, with bracketed Wilson 95% confidence intervals at n=665 (independent-question approximation; see the clustering caveat in Section 5.4). †Hindsight's ingest stage ran under a configuration the benchmark did not control, so its cells are not comparable across the backbone/reasoning columns (Section 5.4).

LoCoMo subset and does not depend on (N, L). The Acc column and its Wilson intervals are therefore identical to Table 3; only $\hat{C}$ and the ratio change. The lowest-cost-per-correct configuration is unchanged: Mastra OM on gpt-oss-20b low ($\hat{C}$/Acc=0.278) and Mem0 on Gemma 4 26B A4B (0.325–0.339). The lower half of the ranking, however, flips. Hindsight is the costliest-per-correct system at $(100, 100)$, but it is no longer last in three of four columns. Mem0 on gpt-oss-20b medium now costs more (2.190 vs. 1.607), as does Mastra OM on both Gemma 4 26B A4B columns (0.768/0.684 vs. 0.543/0.542). The reason is that Hindsight's large fixed ingest cost spreads over a longer conversation, while Mem0 and Mastra OM scale up faster (3.4–5.3× vs. 7–11× from the reference cell; these are $\hat{C}_{(400,200)}/\hat{C}_{(100,100)}$ ratios from Tables 3 and 4 for Hindsight vs. Mem0 and Mastra OM). The full-history figures for gpt-oss-20b are measured only to turn 374 (Section 5.4), so gpt-oss-20b comparisons should be read alongside the break-even statuses in Table 8.

5 Discussion

5.1 Separating Conversation Depth from Message Size

A single cumulative-content predictor $T = N{\cdot}L$ treats two different cases as the same: a long conversation of small messages and a short conversation of large messages. Memory systems handle these two cases differently. We therefore fit message size L and conversation depth t as separate exponents (Table 1). For the window-based baselines the exponents match the known mechanism. The full-history baseline fits $p \approx q \approx 1$, because every token is resubmitted. The rolling-window baseline fits $q \approx 0.11$, so cost is nearly flat in depth; this follows from its bounded window. Held-out error for both stays below 6.5%. Using two predictors instead of one captures this structure: the rolling-window in-sample R^2 rises from 0.39–0.40 under a one-variable power law to 0.91–0.95 here.

The same separable model does *not* hold out for the memory systems. A low in-sample R^2 on its own does not show that a system has separated cost from conversation size. Mastra OM has the highest in-sample R^2 of the three memory systems, yet it has the worst held-out error. The held-out test shows that per-turn

System	Metric	gpt-oss-20b low	gpt-oss-20b medium	Gemma 4 26B A4B low	Gemma 4 26B A4B medium
Mem0	Acc	0.322 [0.287, 0.358]	0.214 [0.184, 0.246]	0.498 [0.460, 0.536]	0.516 [0.478, 0.554]
	$\hat{C}$ (USD)	\$0.435	\$0.469	\$0.169	\$0.167
	$\hat{C}$/Acc	1.350	2.190	0.339	0.325
Mastra OM	Acc	0.361 [0.325, 0.398]	0.308 [0.274, 0.344]	0.429 [0.391, 0.466]	0.502 [0.464, 0.540]
	$\hat{C}$ (USD)	\$0.100	\$0.301	\$0.330	\$0.344
	$\hat{C}$/Acc	0.278	0.979	0.768	0.684
Hindsight†	Acc	0.528 [0.490, 0.565]	0.541 [0.503, 0.579]	0.493 [0.455, 0.531]	0.498 [0.460, 0.536]
	$\hat{C}$ (USD)	\$0.822	\$0.869	\$0.268	\$0.270
	$\hat{C}$/Acc	1.557	1.607	0.543	0.542

Table 4: Joint cost–accuracy matrix at the $(N{=}400, L{=}200)$ cell; all Mem0 and Mastra OM cells, and all Hindsight cells except gpt-oss-20b medium, have crossed break-even against full history (Table 8). $\hat{C}$ is mean billable cost in USD per 400-turn conversation over 8 reps, priced with the same accounting as Table 3 (rates in Appendix A); Acc is identical to Table 3 because it is measured once per cell on the LoCoMo subset and is independent of (N, L), with bracketed Wilson 95% confidence intervals at $n{=}665$ (independent-question approximation; see the clustering caveat in Section 5.4). †Hindsight's ingest stage ran under a configuration the benchmark did not control, so its cells are not comparable across the backbone/reasoning columns (Section 5.4).

cost depends on internal memory state. To predict serving cost for these systems at an unseen workload, we would need a model of the memory subsystem itself. We leave this to future work (Section 5.4).

5.2 Backbone and Reasoning Effort Shape the Cost Surface

The backbone's effect on cost depends on the system and the reasoning level. It does not favor one model in every case. Mem0 shows this most plainly. Its reference-cell cost falls from \$0.059–\$0.065 under gpt-oss-20b to \$0.019 under Gemma 4 26B A4B, and its $(N{=}400, L{=}200)$ cost falls from \$0.435–\$0.469 to \$0.167–\$0.169 (Tables 3 and 4). This gap matches the diagnostics in Table 7. Mem0's answer- and ingest-stage reasoning-to-output ratios are near one on gpt-oss-20b but zero on Gemma 4 26B A4B, and its output-to-input ratio γ exceeds one only on gpt-oss-20b. Mastra OM behaves in the opposite way. Its gpt-oss-20b low cell is the cheapest Mastra OM setting in both joint matrices, which fits its lower γ and lower reasoning ratios than its Gemma 4 26B A4B cells. We exclude Hindsight from cross-backbone comparisons because its ingest stage was not benchmark-controlled (Section 5.4). In practice, backbone and memory choice are not separable: whether a given backbone reduces or increases cost, and by how much, is governed by the memory system's internal call pattern.

Raising reasoning effort from *low* to *medium* increases cost but does not always improve accuracy. Mem0's accuracy on gpt-oss-20b *drops* from 0.322 to 0.214 at the higher reasoning level. This is likely because reasoning tokens use up the `max_tokens` budget and leave less room for the answer. Mastra OM moves in opposite directions on the two backbones (+7.3 pp on Gemma 4 26B A4B, −5.3 pp on gpt-oss-20b). Reasoning effort is therefore not a simple cost knob: at the same step, the accuracy change varies by ≥ 10 percentage points across systems.

5.3 When Does a Memory System Pay Off?

No single system is best in all settings. Mem0's extraction-based ingest keeps its per-turn cost the flattest across (L, t), but its accuracy varies the most of the three (from 0.214 to 0.516). Hindsight is the most expensive system to serve: $\hat{C} \approx 0.24$ USD per 100-turn conversation under gpt-oss-20b, several times any other cell. Mastra OM's threshold-based reflector fires based on internal memory state, so its per-turn cost does not follow conversation length or message size (Section 4.1). The lowest cost-per-correct-answer is Mastra OM on gpt-oss-20b low (0.028 at $(100, 100)$ and 0.278 at $(400, 200)$). Mem0 is the cheapest option on Gemma 4 26B A4B (0.037–0.038 at $(100, 100)$ and 0.325–0.339 at $(400, 200)$). Hindsight on gpt-oss-20b is the most expensive at the reference cell ($\hat{C}$/Acc ≈ 0.45).

The right choice depends on the workload. A memory system is cheaper than full-history serving only after it crosses its break-even length (Section 4.2). Mastra OM breaks even at once, Mem0 within tens of turns once messages are large, and Hindsight only late, sometimes past 400 turns. Whether a memory system is worth its cost therefore depends on the expected conversation length and the chosen backbone, not on the system alone. Full-history cost grows without limit ($p \approx q \approx 1$, Section 4.1) and must eventually fill any finite context window. Within our 400-turn grid, however, no overflow occurs (Section 5.4).

5.4 Limitations

Synthetic cost-benchmark dialogues. The cost benchmark uses LLM-generated dialogues rather than real conversations. This choice is deliberate: billing depends on token counts, which are set by message size and conversation length. Ingest-side behavior, however, can depend on content. Memory fact extraction and memory consolidation rates depend on how many extractable facts appear per turn, and this density may differ in real conversations.

Single accuracy corpus. Accuracy is reported on the LoCoMo subset only. The joint claims hold for the LoCoMo distribution (open-domain persona-grounded multi-session chat) and do not directly transfer to task-oriented or knowledge-intensive QA. Validating the fitted cost model on a task-oriented corpus (MultiWOZ 2.2 [31]) is planned as future work.

Cost model is descriptive, not mechanistic. The cost model in Equation (1) is a regression on conversation length and message size. It generalizes well to the two window-based baselines, but performs poorly on Mem0, Hindsight, and Mastra OM, where leave-one-cell-out errors range from 0.18 to 0.69 (Table 1, Section 5). Cost predictions for these three systems at untested (N, L) values should be read as rough estimates only. We leave a mechanistic model that tracks internal memory state for each system to future work.

Hindsight's ingest backbone was not benchmark-controlled. Hindsight's ingest stage (fact extraction, consolidation, recall ranking) runs in an external self-hosted HTTP server. Its base environment file overrode the per-cell backbone we set, so ingest ran under a single fixed configuration (gpt-oss-20b at a fixed reasoning effort) across all four settings. Only the answer stage, which the harness sets directly, followed the grid. We report Hindsight's per-cell totals (Tables 2, 3, 6 and 7) but exclude it from cross-backbone and reasoning-effort comparisons and from ζ_{ing}. Mem0 and Mastra OM ingest in-process and follow the per-cell configuration (Table 7).

Provider-routing variance. OpenRouter's provider-preference list (⟨ groq, amazon-bedrock, google-vertex ⟩) holds across our runs, but the primary provider can saturate and route requests to a fallback provider. This affects cost in two ways. First, per-token rates differ across providers, so observed billing can differ by single-digit percent from the contract rates used in our fitted costs. Second, prompt caching is tied to one provider. A prefix cached at the primary provider does not hit on the fallback provider, so its tokens are billed at the full input rate ($0.075/M) instead of the cached rate ($0.0375/M, half price). Systems that resend a large fixed context with each answer are most affected. For example, Mastra OM builds up to 40,000 observation tokens. These tokens hit the cached rate on the same provider, but cost twice as much on the input segment when a fallback breaks the cache.

Cost-per-correct-answer is a partial quality metric. The statistic $\hat{C}/\text{Acc}$ in Table 3 ranks systems by cost per LoCoMo-judged correct answer. It treats accuracy as the only quality measure and ignores latency, retrieval-payload size, answer-token budget, retrieval recall, and abstention behavior, any of which can matter in practice. We report the ratio because it follows from the matched cost–accuracy design. We do not claim that minimizing it is the right deployment goal.

Serving-stack token-count mismatch. On gpt-oss-20b, the OpenRouter → groq serving stack rejected full-history answer-stage requests beyond turn 374 of the (N=400, L=200) cell. The rejection cited a 98,516-token prompt, but the true prompt is about 74,800 tokens and fits within the 131,072-token context window. Gemma 4 26B A4B completed all 400 turns of the same conversation. We treat this as a serving-stack error, report gpt-oss-20b full-history costs only to turn 374, and make no context-length claim. Figure 2 extends the curve using the fitted model (held-out error below 6.5%, Section 4.1).

6 Conclusion

This study measures the cost and accuracy of three memory systems under a shared benchmark. We fit a separable cost model ($\log(C{+}1) = a + p\log(L{+}1) + q\log(t{+}1)$) and validate it with leave-one-cell-out cross-validation. Accuracy is scored by a fixed judge on the LoCoMo subset. The main cost finding is that the model predicts well for window-based strategies but not for memory systems. Full-history ($p \approx q \approx 1$) and rolling-window ($q \approx 0$) both hold out below 6.5% error. The retrieval- and threshold-driven memory systems have 18–69% held-out error. This shows that their cost is driven by internal memory state, not by conversation length or message size.

No single system is best in all settings. Mem0's accuracy varies the most (0.214–0.516). Hindsight is the most expensive at the 100-turn reference cell. The lowest cost-per-correct-answer is Mastra OM on gpt-oss-20b low (0.028 at (100, 100) and 0.278 at (400, 200)), while Mem0 leads on Gemma 4 26B A4B (0.037–0.038 at (100, 100) and 0.325–0.339 at (400, 200)). The backbone choice (Gemma 4 26B A4B vs. gpt-oss-20b) shapes the cost surface as much as the system choice, so backbone and memory are a joint decision. Finally, a break-even analysis shows that a memory system becomes cheaper than full-history serving only past a threshold that depends on the workload. This threshold is immediate for the cheapest systems and beyond 400 turns for the most expensive. The choice therefore depends on the expected conversation length, not on the memory system alone.

A System and Backbone Configurations

Table 5 lists the per-system configuration — the ingest trigger, retrieval setting, and the memory-specific parameters the main text names but does not specify values for. Both backbones share temperature 0.7, `max_tokens` 32,768, and per-cell reasoning effort with provider fallbacks disabled, but resolve to different OpenRouter provider stacks: ⟨groq, amazon-bedrock, google-vertex⟩ for gpt-oss-20b and ⟨deepinfra/fp8, io-net/bf16, cloudflare⟩ for Gemma 4 26B A4B. The judge (`gpt-oss-120b`) and the dialogue generator (`gemma-4-26b-a4b-it`, `max_tokens` 8,192) are held fixed across runs. Per-token pricing ($/M tokens) varies by provider. For `gpt-oss-20b`: groq $0.075 (input), $0.0375 (cached input), $0.30 (output); amazon-bedrock $0.07 (input), $0.15 (output); google-vertex $0.07 (input), $0.25 (output). For `gemma-4-26b-a4b-it`: deepinfra/fp8 $0.07 (input), $0.34 (output); io-net/bf16 $0.15 (input), $0.15 (cached input), $0.50 (output); cloudflare $0.10 (input), $0.30 (output). Embeddings use `pplx-embed-v1-0.6b` at $0.004 (embedding input).

B Benchmark Construction and Fitting Procedures

Cost grid. We fit the memory systems on five cells: the four corners and the center, $(N, L) \in \{(15, 50), (15, 200), (100, 100), (400, 50), (400, 200)\}$, at 8 reps each. Mastra OM adds two cells, (800, 200) and (800, 400), at 3 reps to reach its threshold regime. The two deterministic baselines use eight cells at 2 reps: the five above

System	Ingest trigger	Retrieval	Remarks
Mem0	every 10 turns	top-k=10	–
Hindsight	every 10 turns	top-k=10	Ingest-stage LLM is *not* benchmark-controlled; recall budget *mid*, max_tokens=4,096.
Mastra OM	token threshold	top-k=1 over observations	observer is triggered at 30,000 accumulated message tokens, reflector at 40,000 accumulated observation tokens.
Full history	every turn	— (full transcript)	–
Rolling window (k=10)	every turn	— (last 10 turns)	–

Table 5: Per-system configuration. The benchmark controls the answer-stage LLM for all systems and the ingest-stage LLM for Mem0 and Mastra OM; Hindsight's ingest LLM is not benchmark-controlled (Section 5.4).

plus three matched-content cells, $(40, 250)$, $(200, 50)$, $(400, 25)$. Each cost fit uses every turn of every run: 40 runs for Mem0 and Hindsight, 46 for Mastra OM, and 16 per baseline.

Synthetic dialogue generation. For each (N, L, seed), the generator runs in chunks of 20 turns, using the chunk prompt in Appendix C. We tokenize each turn with `o200k_base` [22]. Turns above $1.15{\cdot}L$ are trimmed and turns below $0.85{\cdot}L$ are padded with filler words (tolerance 0.15). We cache each completed dialogue on disk under its (N, L, seed) key, so the conversation is the same across reps and across systems.

LoCoMo subset selection. We score candidate four-dialogue subsets by the Jensen–Shannon divergence [18] between the subset and the full dataset, measured across question category, evidence count, and evidence span. We keep the subset with the lowest total divergence ($< 3 \cdot 10^{-5}$ per axis). Following dataset convention, we exclude Category-5 adversarial questions, which leaves 665 evaluated QA pairs.

Cost-model fitting. We compute one OLS fit of Equation (1) per (system, setting), with $\log(C{+}1)$ as the response. The 95% confidence intervals for the exponents come from a cluster bootstrap that resamples whole runs with replacement (B=1,000). LOOCV-MAPE holds out one (N, L) cell at a time, refits on the remaining cells, and predicts the held-out cell's mean per-turn cost. The number of folds per system equals the number of cells (8 for the baselines, 7 for Mastra OM, 5 for Mem0 and Hindsight). For the memory systems, the held-out errors cluster at the short-conversation and small-message grid corners (Section 4.1). Table 6 gives the per-(system, setting) fits that Table 1 summarizes, and Table 7 gives the token-accounting diagnostics cited in Section 4.1 and Section 5.

Break-even computation. We compute the break-even turn from measured per-turn cost (8 reps averaged), with no fitted-model extrapolation. For each system and for the full-history baseline, we add up per-turn cost over depth. The break-even turn is the smallest t at which the system's cumulative cost falls below full-history's and *stays* below it at every later turn (*immediate* = cheaper from turn 0; *never* = still costlier at the last measured turn). Table 8 reports it for all measured multi-system cells. For gpt-oss-20b full-history at (N=400, L=200), the run ended at turn 374 because of a serving-stack error (Section 5.4), so we report full-history cost only to that turn.

C Prompt Templates

The two boxes below give the LLM-as-judge and dialogue-generation prompts verbatim. Answer-stage prompts are system-specific — each instructs a brief answer grounded only in the retrieved memory or transcript, with relative time references resolved to absolute dates from message timestamps. We used each system's default prompt without modification; readers should refer to the respective cited papers and system documentation for those templates.

LLM-as-Judge Prompt (`gpt-oss-120b`)

System. You are an expert grader that determines if answers to questions match a gold standard answer.
User. Your task is to label an answer to a question as 'CORRECT' or 'WRONG'. You will be given the following data: (1) a question (posed by one user to another user), (2) a 'gold' (ground truth) answer, (3) a generated answer, which you will score as CORRECT/WRONG.
The point of the question is to ask about something one user should know about the other user based on their prior conversations. The gold answer will usually be a concise and short answer that includes the referenced topic. The generated answer might be much longer, but you should be generous with your grading — as long as it touches on the same topic as the gold answer, it should be counted as CORRECT.
For time-related questions, the gold answer will be a specific date, month, year, etc. The generated answer might be much longer or use relative time references (e.g. "last Tuesday" or "next month"), but you should be generous with your grading — as long as it refers to the same date or time period as the gold answer, it should be counted as CORRECT. Even if the format differs (e.g. "May 7th" vs. "7 May"), consider it CORRECT if it is the same date.
Question: `{question}` **Gold answer:** `{golden_answer}` **Generated answer:** `{generated_answer}`
First, provide a short (one sentence) explanation of your reasoning, then finish with CORRECT or WRONG. Do NOT include both CORRECT and WRONG in your response. Return the label in JSON format with the key `"label"`.

Synthetic Dialogue Generation — Chunk Prompt

Generate `{chunk_size}` turns of a natural two-speaker personal conversation. Each turn should be about `{l_tokens}` raw tokens long under the o200k_base tokenizer. Keep facts concrete: names, preferences, routines, dates, plans, relationships. Return JSON only, as an object with key `"turns"` whose value is an array of strings. No markdown. Seed: `{seed}`. First turn index: `{start_idx}`.

Disclosure of Generative AI Usage

In preparing this work, the authors used Claude Code (Opus 4.7 and Sonnet 4.6) as an assistive tool. The research questions, the benchmark design, the choice of memory systems, backbones, and

System	Setting	p (L)	q (t)	R^2	LOOCV-MAPE	max APE
Full history	Gemma 4 26B A4B low	0.97	0.97	0.999	0.029	0.089
Full history	Gemma 4 26B A4B medium	0.97	0.97	0.999	0.030	0.089
Full history	gpt-oss-20b low	0.95	0.94	0.996	0.053	0.145
Full history	gpt-oss-20b medium	0.95	0.94	0.996	0.053	0.145
Rolling window (k=10)	Gemma 4 26B A4B low	0.91	0.11	0.913	0.062	0.148
Rolling window (k=10)	Gemma 4 26B A4B medium	0.92	0.12	0.945	0.061	0.142
Rolling window (k=10)	gpt-oss-20b low	0.85	0.10	0.949	0.065	0.128
Rolling window (k=10)	gpt-oss-20b medium	0.85	0.10	0.950	0.065	0.128
Mem0	Gemma 4 26B A4B low	0.18	0.08	0.069	0.184	0.244
Mem0	Gemma 4 26B A4B medium	0.16	0.09	0.067	0.185	0.254
Mem0	gpt-oss-20b low	0.16	0.07	0.048	0.222	0.282
Mem0	gpt-oss-20b medium	0.18	0.08	0.064	0.200	0.243
Hindsight	Gemma 4 26B A4B low	0.14	0.43	0.348	0.477	0.555
Hindsight	Gemma 4 26B A4B medium	0.15	0.43	0.345	0.478	0.558
Hindsight	gpt-oss-20b low	0.15	0.41	0.328	0.476	0.569
Hindsight	gpt-oss-20b medium	0.17	0.41	0.320	0.461	0.570
Mastra OM	Gemma 4 26B A4B low	0.61	0.23	0.457	0.408	0.820
Mastra OM	Gemma 4 26B A4B medium	0.68	0.25	0.458	0.685	1.918
Mastra OM	gpt-oss-20b low	0.79	0.36	0.557	0.477	0.864
Mastra OM	gpt-oss-20b medium	0.79	0.34	0.520	0.565	1.596

Table 6: Per-(system, setting) fits of the separable cost model $\log(C{+}1) = a + p\log(L{+}1) + q\log(t{+}1)$**. R^2 is in-sample; LOOCV-MAPE and max APE are leave-one-(N, L)-cell-out held-out errors. Full-history holds out 8 folds per row, rolling-window 8, Mastra OM 7, Mem0 5, Hindsight 5 (all five (N, L) cells are completed for Hindsight under every setting). Table 1 summarizes these rows as ranges per system.**

evaluation metrics, and the analysis and interpretation of the results represent the authors' own novel intellectual contributions. Within that direction, the tool was used to (i) support brainstorming and refinement of framing; (ii) write and debug the benchmarking and analysis code that produced the results; and (iii) draft and revise the text of the manuscript, including tables and figure captions. The authors reviewed and verified all generated content—confirming the correctness of the code, the validity of the experimental results, and the accuracy of all claims and citations—and edited the text as needed. The authors take full responsibility for the veracity and correctness of the entire content of this work.

System	Setting	γ	ζ_{ans}	ζ_{ing}
Hindsight	Gemma 4 26B A4B low	0.16	0.00	–
Hindsight	Gemma 4 26B A4B medium	0.16	0.00	–
Hindsight	gpt-oss-20b low	0.16	1.01	–
Hindsight	gpt-oss-20b medium	0.16	1.00	–
Mem0	Gemma 4 26B A4B low	0.56	0.00	0.00
Mem0	Gemma 4 26B A4B medium	0.55	0.00	0.00
Mem0	gpt-oss-20b low	1.27	0.99	0.93
Mem0	gpt-oss-20b medium	1.40	1.01	0.89
Mastra OM	Gemma 4 26B A4B low	0.43	1.03	0.83
Mastra OM	Gemma 4 26B A4B medium	0.44	1.02	0.81
Mastra OM	gpt-oss-20b low	0.23	0.36	0.31
Mastra OM	gpt-oss-20b medium	0.53	0.96	0.74

Table 7: Per-stage token-accounting diagnostics: output-to-input token ratio γ, answer-stage reasoning-to-output ratio ζ_{ans}, and ingest-stage reasoning-to-output ratio ζ_{ing}, computed from the per-cell run logs. Each ζ is the ratio of reasoning (thinking) tokens to visible-response tokens reported by the API; values above 1 occur when the model spends more tokens on internal reasoning than on its visible response, which is possible when the API reports them as separate counts. The two baselines incur no ingest-stage LLM cost and are omitted. Hindsight's ζ_{ing} is omitted (–): its ingest-stage LLM ran under a configuration the benchmark did not control (Section 5.4).

Setting	Cell (N×L)	Mem0	Hindsight	Mastra OM
Gemma 4 26B A4B low	100×100	10	never	54
	400×50	23	223	86
	400×200	0	60	50
Gemma 4 26B A4B medium	100×100	0	never	54
	400×50	21	224	73
	400×200	0	61	68
gpt-oss-20b low	100×100	82	never	0
	400×50	260	never	0
	400×200	82	356	0
gpt-oss-20b medium	100×100	never	never	13
	400×50	342	never	62
	400×200	40	never	28

Table 8: Sustained break-even turn per (setting, cell, system). *never* = the system is still costlier than the full transcript at the last measured turn. By turn 400 the full transcript costs up to 12.7× a system that has broken even (maximum ratio of full-history to adapter cumulative cost at the last measured turn, over all setting/cell/adapter combinations where the adapter has crossed break-even; attained by Mastra OM on gpt-oss-20b low at the 400×50 cell). Hindsight at small messages costs up to 3.3× the full transcript (maximum ratio of Hindsight to full-history cumulative cost over all 400-turn *never* cells; attained on gpt-oss-20b medium at the 400×50 cell).

References

[1] Anthropic. 2024. Prompt Caching. https://platform.claude.com/docs/en/build-with-claude/prompt-caching.

[2] Anthropic. 2025. Claude API Pricing. https://platform.claude.com/docs/en/about-claude/pricing.

[3] Tyler Barnes. 2026. Observational Memory: 95% on LongMemEval. Mastra Technical Report. https://mastra.ai/research/observational-memory

[4] Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. 2001. Interval Estimation for a Binomial Proportion. *Statist. Sci.* 16, 2 (2001), 101–133. doi:10.1214/ss/1009213286

[5] A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. 2008. Bootstrap-Based Improvements for Inference with Clustered Errors. *The Review of Economics and Statistics* 90, 3 (2008), 414–427. doi:10.1162/rest.90.3.414

[6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413 [cs.CL] https://arxiv.org/abs/2504.19413

[7] B. Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. doi:10.1214/aos/1176344552

[8] Sedigheh Eslami, Maksim Gaiduk, Markus Krimmel, Louis Milliken, Bo Wang, and Denis Bykov. 2026. Diffusion-Pretrained Dense and Contextual Embeddings. arXiv:2602.11151 [cs.LG] https://arxiv.org/abs/2602.11151

[9] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. arXiv:2311.04934 [cs.CL] https://arxiv.org/abs/2311.04934

[10] Google DeepMind. 2026. Gemma 4 Model Card. https://ai.google.dev/gemma/docs/core/model_card_4

[11] Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, Radha Poovendran, Gregory Wornell, Lyle Ungar, Dan Roth, Sihao Chen, and Camillo Jose Taylor. 2025. PersonaMem-v2: Towards Personalized Intelligence via Learning Implicit User Personas and Agentic Memory. arXiv:2512.06688 [cs.CL] https://arxiv.org/abs/2512.06688

[12] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. arXiv preprint arXiv:2310.06839. arXiv:2310.06839 [cs.CL] https://arxiv.org/abs/2310.06839

[13] Maria Khalusova. 2024. RAG vs. Long-Context Models: Do We Still Need RAG? https://unstructured.io/blog/rag-vs-long-context-models-do-we-still-need-rag.

[14] Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. 2025. Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects. arXiv:2512.12818 [cs.CL] https://arxiv.org/abs/2512.12818

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[16] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

[17] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753 [cs.CL] https://arxiv.org/abs/2402.17753

[18] M.L. Menéndez, J.A. Pardo, L. Pardo, and M.C. Pardo. 1997. The Jensen–Shannon divergence. *Journal of the Franklin Institute* 334, 2 (1997), 307–318. doi:10.1016/S0016-0032(96)00063-4

[19] OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[20] OpenAI. 2024. Prompt Caching in the API. https://openai.com/index/api-prompt-caching/.

[21] OpenAI. 2025. OpenAI API Pricing. https://platform.openai.com/docs/pricing.

[22] OpenAI. 2025. tiktoken: A Fast BPE Tokeniser for Use with OpenAI's Models. https://github.com/openai/tiktoken.

[23] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] https://arxiv.org/abs/2310.08560

[24] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956 [cs.CL] https://arxiv.org/abs/2501.13956

[25] Jeffrey Rengifo and Eduard Martin. 2025. Longer Context ≠ Better: Why RAG Still Matters. https://www.elastic.co/search-labs/blog/rag-vs-long-context-model-llm.

[26] M. Stone. 1974. Cross-Validatory Choice and Assessment of Statistical Predictions. *Journal of the Royal Statistical Society: Series B (Methodological)* 36, 2 (1974), 111–147. doi:10.1111/j.2517-6161.1974.tb00994.x

[27] Miao Su, Yucan Guo, Zhongni Hou, Long Bai, Zixuan Li, Yufei Zhang, Guojun Yin, Wei Lin, Xiaolong Jin, Jiafeng Guo, and Xueqi Cheng. 2026. Beyond Dialogue Time: Temporal Semantic Memory for Personalized LLM Agents. arXiv:2601.07468 [cs.AI] https://arxiv.org/abs/2601.07468

[28] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953

[29] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813 [cs.CL] https://arxiv.org/abs/2410.10813

[30] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110 [cs.CL] https://arxiv.org/abs/2502.12110

[31] Xiaoxue Zang, Abhinav Rastogi, Srinivas Sunkara, Raghav Gupta, Jianguo Zhang, and Jindong Chen. 2020. MultiWOZ 2.2: A Dialogue Dataset with Additional Annotation Corrections and State Tracking Baselines. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*. Association for Computational Linguistics, Online, 109–117. https://aclanthology.org/2020.nlp4convai-1.13